CENG-567
REINFORCEMENT LEARNING

Lecture-2

Multi-armed Bandits (Contd.)

Upper Confidence Bounds (UCB), Thompson Sampling

Dr. Osman GÖKALP

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap

1. Multi-Armed Bandit (MAB) fundamentals

Simplified form of Reinforcement Learning

Single-step decision problem (no future states)

Goal: maximize immediate reward

2. Exploration–Exploitation Trade-Off

Exploration: try new actions to gather information

Exploitation: use known best actions to gain reward

Key challenge: balancing both efficiently

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap

3. Exploration strategies

A/B/n Testing:

Fixed test phase, static probabilities
Simple but inefficient and non-adaptive

ε-Greedy:

With prob. (1–ε): exploit best action
With prob. (ε): explore randomly
Better adaptability, still somewhat static

Key takeaway:
Finding the right balance between **exploration** and **exploitation** is essential for optimal decision-making in MAB problems.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Action Selection Using Upper Confidence Bounds (UCB)

Key Idea:
Upper Confidence Bounds (UCB) is a principled method to balance **exploration** and **exploitation**.

Concept:
At each time step, choose the action with the **highest potential reward**, defined as:

Potential = Estimated Value + Uncertainty

An action may be selected because:

Its estimated reward is high, **or**

It is **less explored**, hence more uncertain, **or both**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

The UCB Formula

We select the action to take at time t that using the following formula:

$$A_t \triangleq \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right]$$

Where:

$Q_t(a)$:estimated action value

$N_t(a)$:number of times action a was selected

t: current time step

c: hyperparameter controlling exploration strength

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Understanding the Formula

Breakdown:

$Q_t(a)$:**exploitation term** → favors actions with high average reward

$\sqrt{\frac{\ln t}{N_t(a)}}$:**exploration term** → favors uncertain actions

As $N_t(a)$ → ↑uncertainty ↓

As t↑ → overall uncertainty ↑ slightly → encourages continued exploration

Tunable parameter:

$\boldsymbol{c}$ adjusts how aggressively uncertainty is explored

Must be tuned carefully; **too low** → under-exploration, **too high** → over-exploration

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Online advertising scenario with UCB

- See '**Multi-armed bandits.ipynb**' file in the Teams course page.

Setup:

Define ads and initialize parameters

At each impression:

Compute the UCB for each ad
Select the ad with the highest UCB
Observe reward (click / no click)
Update counts and estimates

If an action has not been selected yet, it has the highest UCB.

Action Selection Logic

Key behavior of UCB:

Unseen ads → highest uncertainty → selected early

Well-tested ads → lower uncertainty → less exploration

Exploration automatically decreases as more impressions accumulate

Tie-breaking:

If multiple ads have the same UCB → choose randomly

✅ *Efficient exploration without fixed probabilities (unlike ε-greedy).*

Hyperparameter c and Its Effect

c: exploration coefficient

Controls how strongly uncertainty affects decision-making

Low c→ more exploitation

High c→ more exploration

Practical note:

Must be tuned empirically

Performance varies significantly with c

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example Output

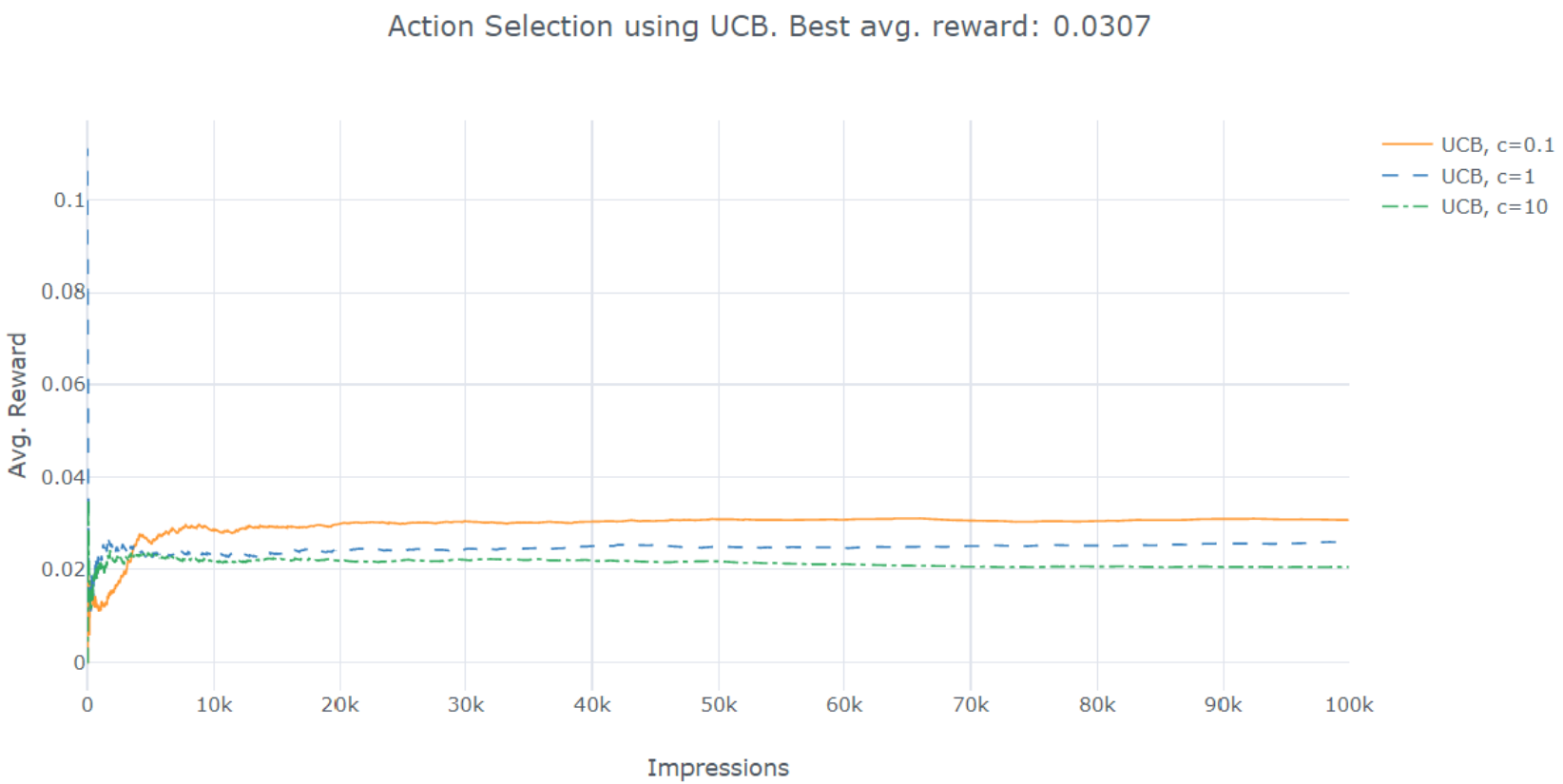


Figure 2.4 – Exploration using UCB

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Meta Observation – Hyperparameter Tuning

Challenge:
Tuning c is **itself a bandit problem**!

Approach:

Try logarithmic scales first: [0.01, 0.1, 1, 10]

Once a good range is found, refine linearly

💡 *In practice, once you find a good c for a specific domain (e.g., online ads), it often generalizes to similar tasks.*

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Advantages and disadvantages of using UCBs

- **UCB is a set-and-forget approach**. It systematically and dynamically allocates the budget to alternatives that need exploration.

- **UCB can be further optimized for dynamic environments, potentially at the expense of introducing additional hyperparameters**. The formula we provided for UCB is a common one, but it can be improved – for example, by using exponential smoothing to calculate the Q values. There are also more effective estimations of the uncertainty component in literature.

- **UCB could be hard to tune**. It is somewhat easier to make the call and say, "I want to explore 10% of the time, and exploit for the rest" for the ε-greedy approach than saying, "I want my to be 0.729" for the UCB approach, especially if you are trying these methods on a brand-new problem.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

UCB in Dynamic Environments: Exponential Smoothing

Goal: Make UCB responsive to **changing reward distributions**.

Key Idea: Update the estimated action value $Q(a)$using **recent rewards more heavily**.

Standard update:

$$Q_{t+1}(a) = Q_t(a) + \frac{1}{N(a)}\left(R_t - Q_t(a)\right)$$

Exponential smoothing update:

$$Q_{t+1}(a) = (1-\alpha)Q_t(a) + \alpha R_t$$

$\alpha \in (0, 1]$ =smoothing factor (large → fast adaptation, small → smoother estimate)

Selection formula (argmax) **remains the same.**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Thompson (Posterior) Sampling

- Goal in Multi-Armed Bandit (MAB) problems:
  Estimate the parameter(s) of the reward distribution for each arm (e.g., which ad to display). Measure uncertainty about estimate to guide the exploration strategy.

- This problem fits naturally into the **Bayesian inference** framework.

- Bayesian inference **starts with a prior** probability distribution – an initial idea, for the parameter θ – and **updates this prior distribution as data becomes** available.

- So, the Bayesian approach **treats the parameter as a random variable** given the data.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Bayesian Inference for Thompson Sampling

Start with a **prior distribution** (initial guess for the parameter).

Update prior with observed **data** → obtain **posterior distribution**.

Bayesian interpretation: Treat the **parameter as a random variable** given the data.

Formula: Posterior: $P(\theta \mid X) = \frac{P(X|\theta)P(\theta)}{P(X)}$

X : Data

$P(\theta)$:prior

$P(X \mid \theta)$:likelihood

$P(\theta \mid X)$:posterior

$P(X)$:evidence

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

θ is model-dependent

- **Bernoulli**: $\theta = p$ (probability of success).
- **Normal** with **known variance**: $\theta = \mu$.
- **Normal** with unknown mean & variance: $\theta = (\mu, \sigma^2)$.
- **Poisson**: $\theta = \lambda$ (rate).
- **Logistic regression / contextual bandit**: $\theta = \beta$ (vector of coefficients).

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Thompson Sampling in Online advertising scenario

- Each **ad** corresponds to an **arm**.
- A **click** is a **Bernoulli random variable** (1 = click, 0 = no click).
- Goal: Estimate the **click-through rate (CTR)** = probability of a click.
- CTR $\in$ [0, 1], same logic applies to any **binary-outcome** problem.
- Note that many problems other than online advertising have such a binary outcome. So, this discussion here can be extended to such other cases.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Initial Beliefs (Prior Distribution)

Initially, we have **no reason** to believe one ad's CTR is higher than another.

Therefore, assume a **uniform prior** for each ad's CTR:

$$\theta_k \sim \text{Uniform}(0,1)$$

or equivalently

$$\theta_k \sim \text{Beta}(1,1)$$ *(we'll discuss beta distribution later)*

Each θ_k represents the **true CTR** of ad k, which we are uncertain about.

As data arrive, we'll update this belief.

Updating Beliefs with Observations

When ad k is displayed and **clicked (1)** → update posterior toward 1.

When ad k is displayed and **not clicked (0)** → update posterior toward 0.

Over time:

- **Expected value** of θ_k shifts based on observed clicks.
- **Variance** shrinks → less uncertainty.

Connection to Exploration vs Exploitation:

High variance → more exploration (less data).

Low variance → more exploitation (well-known ad).

Same principle as **UCB**, but derived via **Bayesian inference**.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Sampling Step – Choosing the Ad

At each time step:

1. For each ad k, draw one sample from its posterior:
$$\tilde{\theta}_k \sim P(\theta_k \mid X)$$
2. Select the ad with the **highest sampled** $\tilde{\theta}_k$.
3. Observe the outcome (click or not), then **update** that ad's posterior.

Intuition:

Ads with high expected CTR → more likely to win samples (exploitation).

Ads with high uncertainty (wide posterior) → produce diverse samples (exploration).

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Update Rules for the Bernoulli Distribution

Now, let's talk about how we update our beliefs when we observe new data.

A common choice for the **prior** is the **Beta distribution**.

Think about it — our unknown parameter θ, which represents the click-through rate (CTR), can only take values between **0 and 1**. The Beta distribution also lives on this interval, so it's a perfect match.

Update Rules for the Bernoulli Distribution

When we combine this **Beta prior** with a **Bernoulli likelihood** (that is, each ad either gets a click or not), something very convenient happens:

The **posterior** also becomes a **Beta distribution**.
This means after each observation, we simply update the two parameters of the Beta distribution — one for successes (clicks) and one for failures (no-clicks).

This property is called **conjugacy**, and the Beta distribution is known as the **conjugate prior** for the Bernoulli distribution.

Because of this, we can easily perform **sequential updates**:
each time a new click or no-click is observed, we just tweak our Beta parameters and carry on — no complicated recalculations needed.

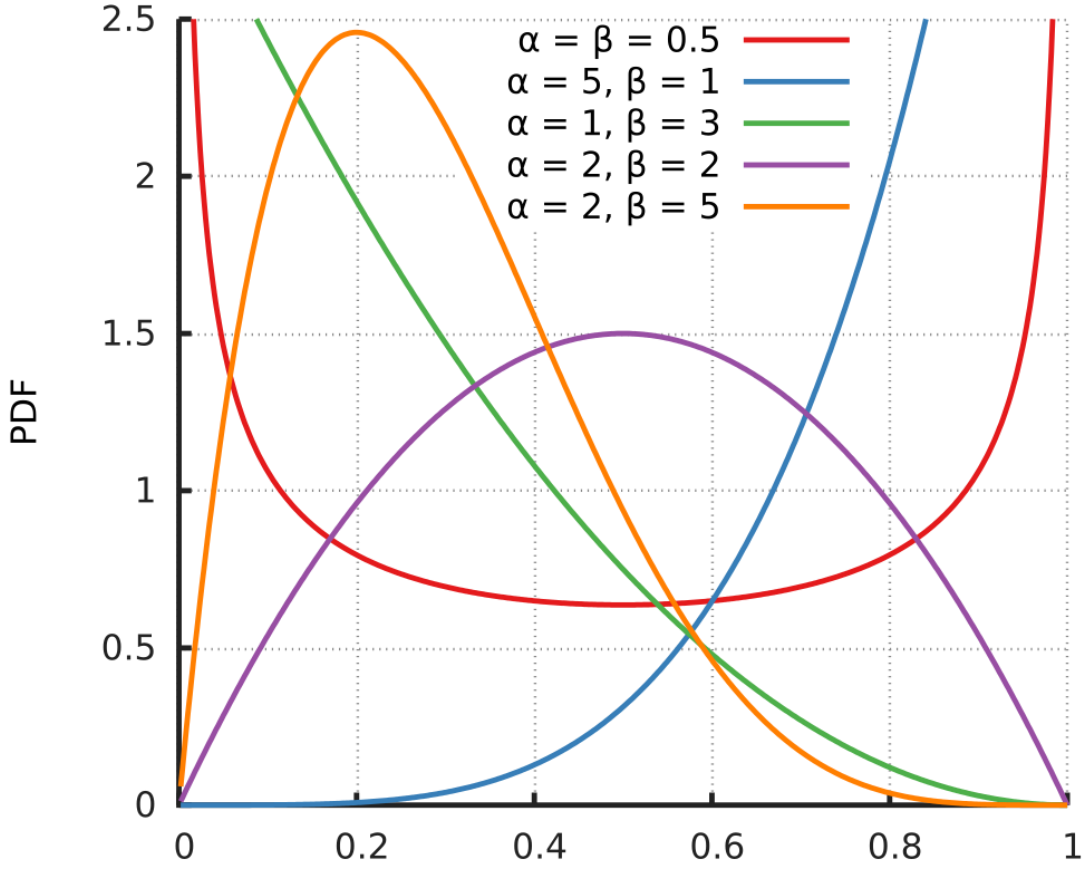
In short:
We model our uncertainty about CTRs with Beta distributions, update them as new data arrive, and always stay within the same distribution family — a simple yet powerful mechanism behind Thompson Sampling.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Beta Distribution- Probability Density Function

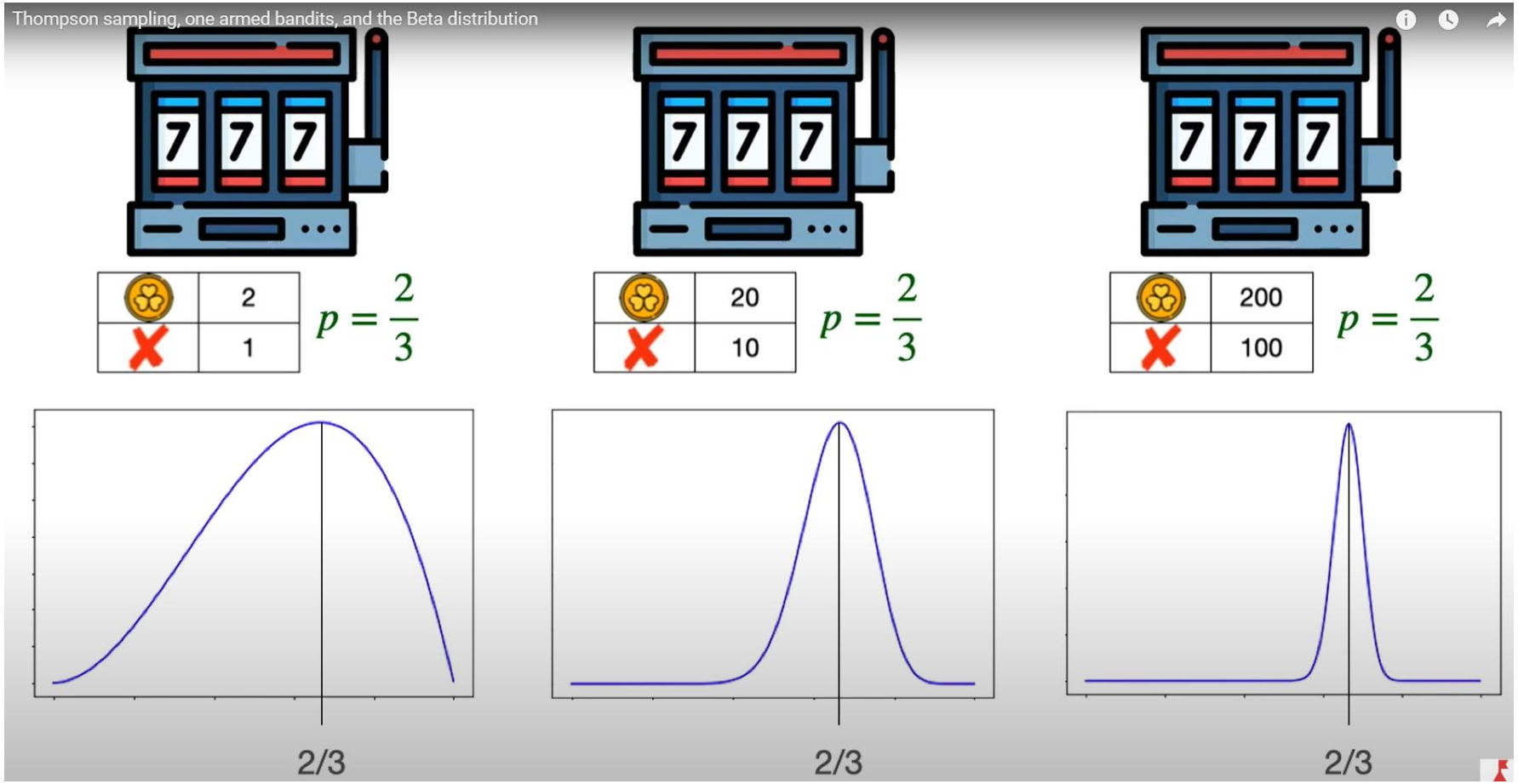
The parameters **α (alpha)** and **β (beta)** are what *shape* the Beta distribution.

Samples from a Beta distribution **represent** possible probabilities reflecting our **beliefs** about an **unknown binary event's chance of success**.



Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example Beta Distributions



Src: https://www.youtube.com/watch?v=nkyDGGQ5h60

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example Beta Distributions

Use for scaling: make AUC 1

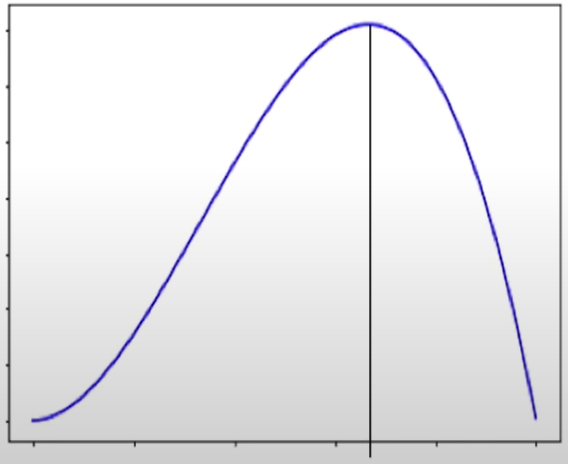
# of success a

# of failure b

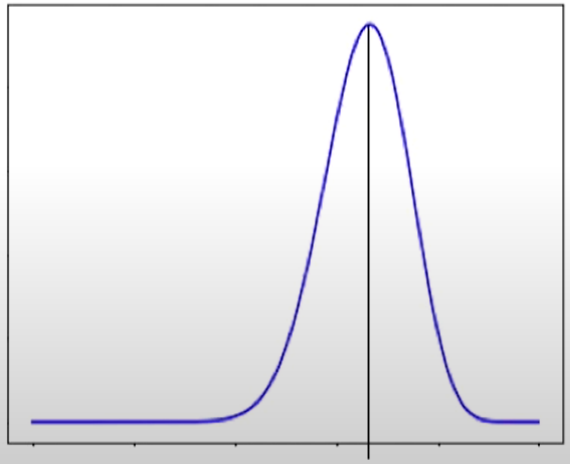
$$f(x) = x^a \cdot (1-x)^b \cdot \boxed{\frac{1}{B(a+1,b+1)}}$$

$$Beta(a+1, b+1)$$

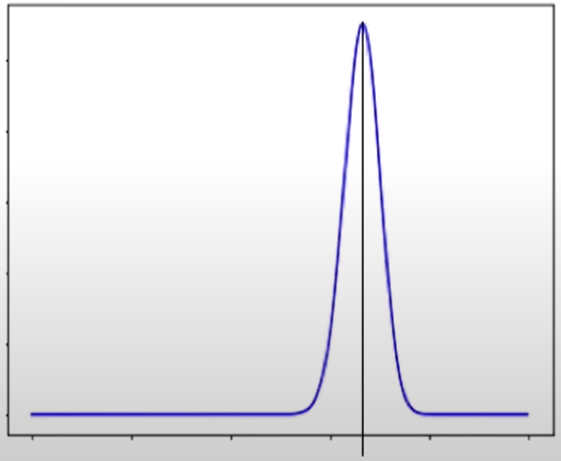
$$f(x) = x^2 \cdot (1-x)^1 \cdot \frac{1}{B(3,2)}$$



$$f(x) = x^{20} \cdot (1-x)^{10} \cdot \frac{1}{B(21,11)}$$



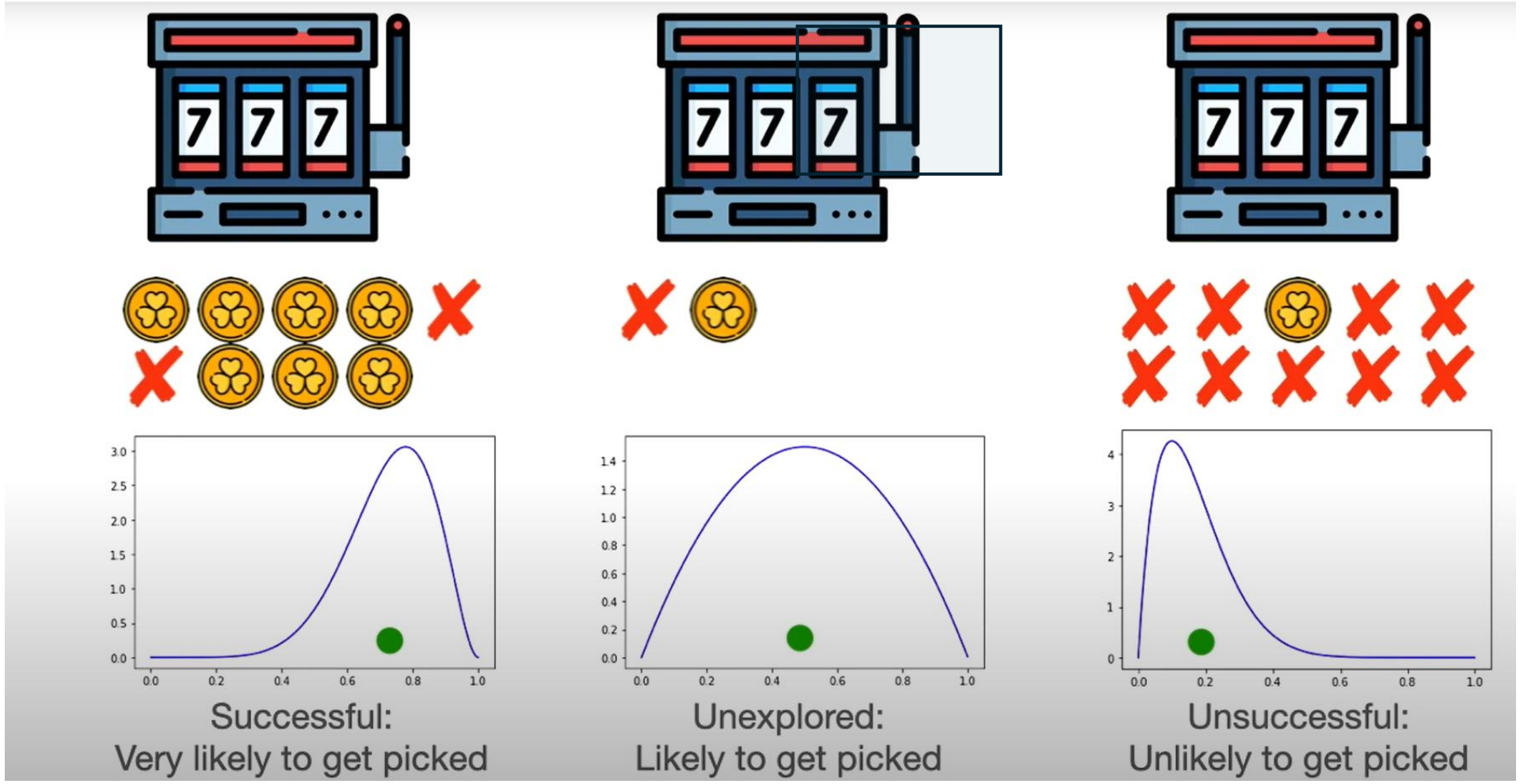
$$f(x) = x^{200} \cdot (1-x)^{100} \cdot \frac{1}{B(201,101)}$$



Src: https://www.youtube.com/watch?v=nkyDGGQ5h60

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example Beta Distributions



Src: https://www.youtube.com/watch?v=nkyDGGQ5h60

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

How to do this in Python?

Count number of success and failure:

- $\alpha_k \leftarrow \alpha_k + R_t$
- $\beta_k \leftarrow \beta_k + (1 - R_t)$

Where, R_t is the reward (1 or 0)

Use the following simple command (numpy):

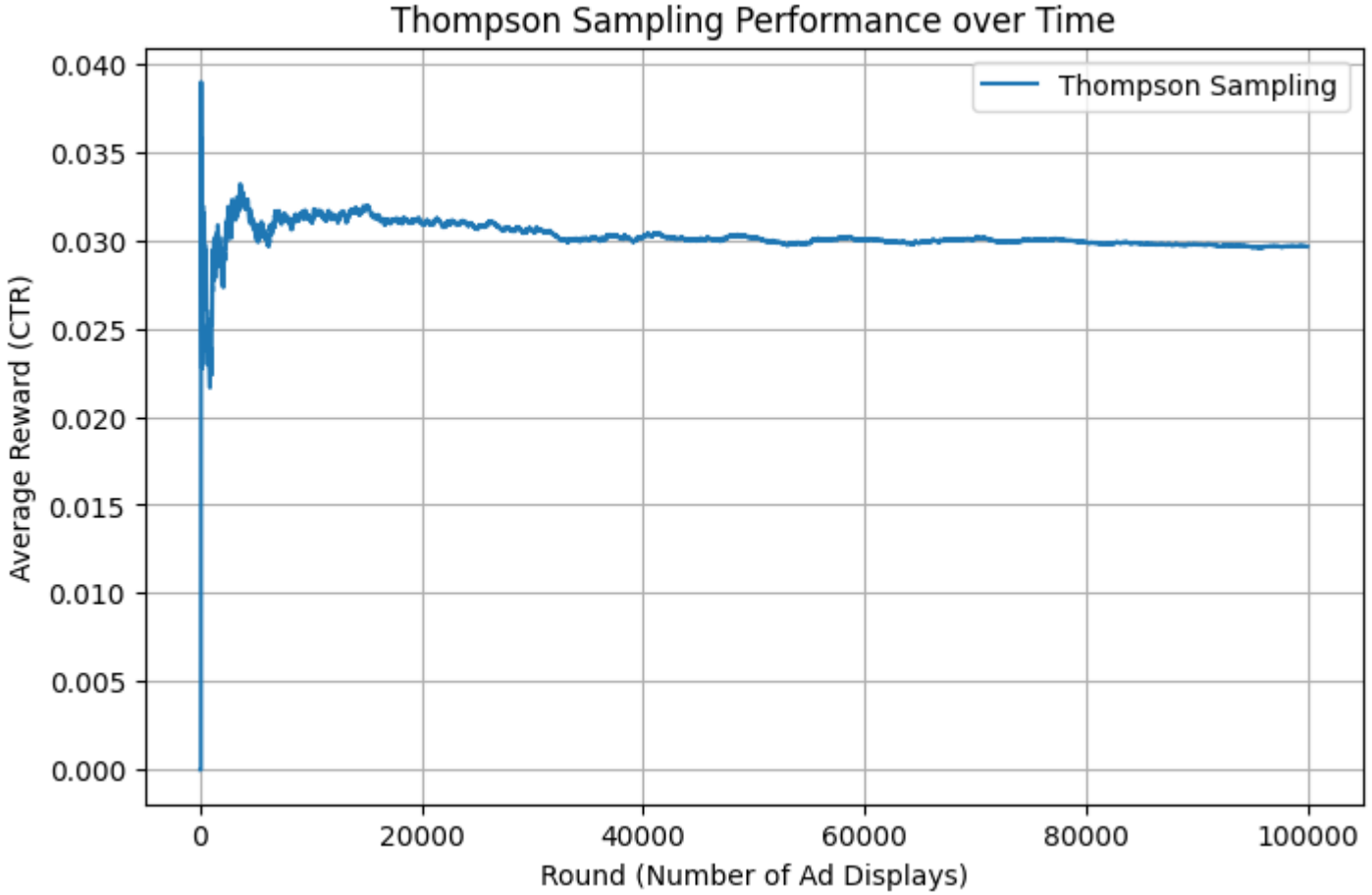
Suppose alphas[k] equals α_k, and betas[k] equals β_k;

```
sample = np.random.beta(alphas[k], betas[k])
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

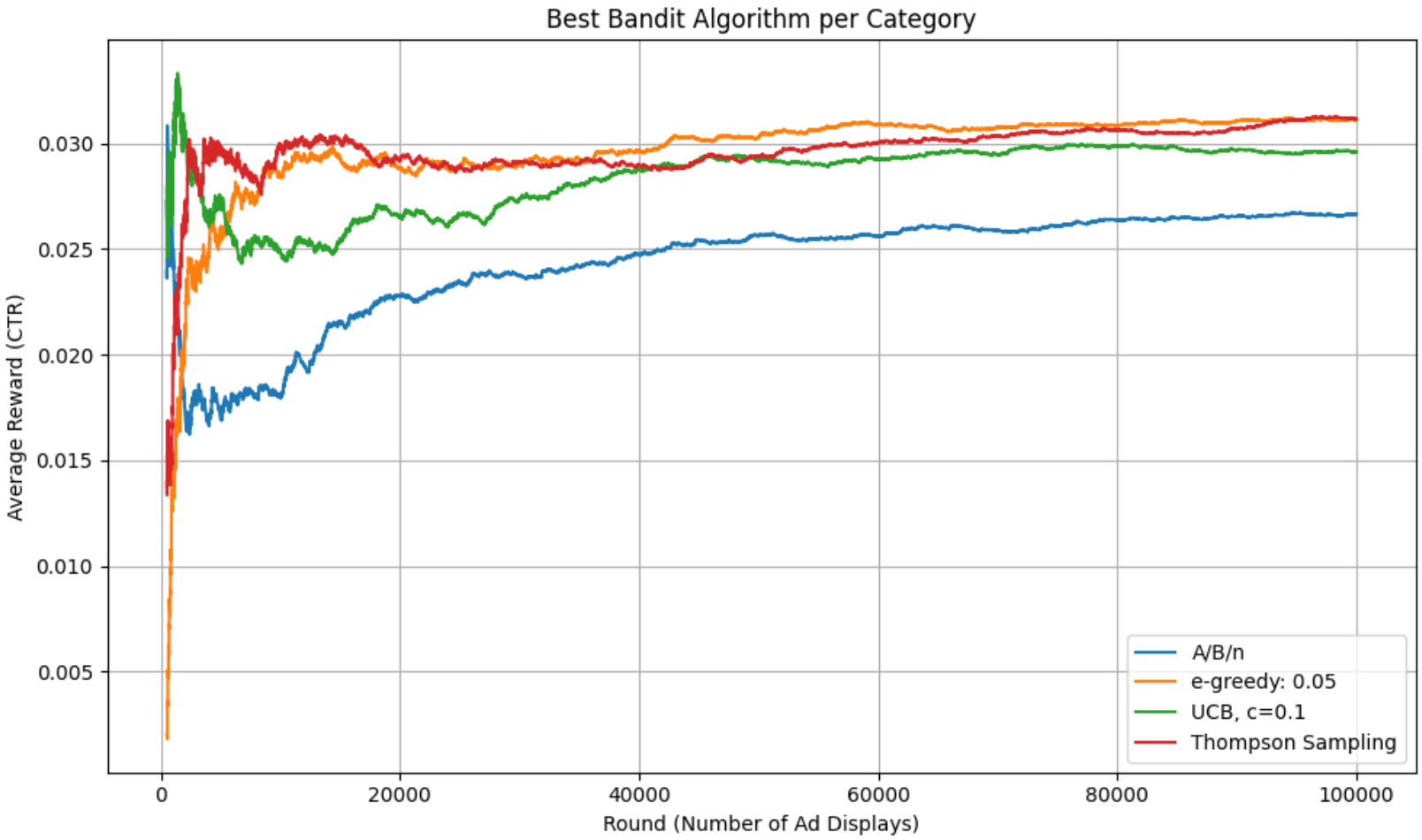
Online Advertising Scenario with Thompson Sampling

- See '**Multi-armed bandits.ipynb**' file in the Teams course page.



Comparing Algorithms

Note that different plots are possible after each run due to stochastic nature of the algorithms



Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Next week

Contextual Bandits (CBs)

- CBs allow us to leverage context information (for instance, the user persona, age, gender, location, or previous visits), which means, they play a role in central role in commercial personalization and recommendation applications.